

CYBER SECURITY INTERNSHIP

TASK 1 – PASSWORD STRENGTH CHECKER

Project Report

Organization: CodeOrbit Tech

Technology: Python 3

Student Name:- Mohini Vijay Pawar

College / Institute:-SSVPS College Of Engineering Dhule

Internship Duration:-1 Month

1. INTRODUCTION

Password security is an important part of cybersecurity. Weak or predictable passwords can make accounts easier to compromise. The Password Strength Checker is a beginner-friendly Python application designed to evaluate a password using common composition rules.

2. PROJECT OBJECTIVE

The objective of this project is to develop a Python program that checks a password for minimum length, uppercase letters, lowercase letters, digits and special characters, then provides a simple strength classification and improvement guidance.

3. PROBLEM STATEMENT

Users often create passwords that are short or use only one type of character. Such passwords may provide poor security. This project provides a simple educational method to identify missing password characteristics.

4. TECHNOLOGY USED

- Python 3
- Python Regular Expression module (re)
- Python IDLE / VS Code / any Python-compatible editor

5. REQUIREMENTS

Hardware: Any computer capable of running Python.

Software: Python 3 and a Python editor or IDLE.

6. WORKING PRINCIPLE

The program accepts a password from the user. It then performs five checks: length, uppercase letter, lowercase letter, number and special character. Each satisfied condition adds one point. The total score is used to classify the password as Weak, Medium or Strong.

7. ALGORITHM

1. Start the program.
2. Accept password input.
3. Check whether length is at least 8 characters.
4. Check uppercase letters.
5. Check lowercase letters.
6. Check digits.
7. Check special characters.
8. Calculate score.
9. Display password analysis and strength.
10. Stop.

8. FEATURES

- Checks password length.
- Detects uppercase letters.
- Detects lowercase letters.
- Detects numbers.
- Detects special characters.
- Classifies password strength.
- Gives suggestions for improvement.

9. SAMPLE TEST CASES

Input	Expected Result	Reason
Mohini	Weak	Short and missing character types
Mohini1234	Medium	Mixed letters and numbers
Mohini@1234	Strong	All major character types are present

10.Source Code

```
import re

print("=====
===") print("      PASSWORD STRENGTH
CHECKER")

print("=====
===")

password = input("Enter your password: ").
score = 0.
if len(password) >= 8:

    score += 1  if

re.search(r"[A-Z]", password):

    score += 1  if

re.search(r"[a-z]", password):

    score += 1  if

re.search(r"[0-9]", password):

    score += 1  if re.search(r"^[A-Za-
z0-9]", password):

    score += 1

print("\nPassword
Analysis:")

if len(password) >= 8:

print("Length: Good") else:

print("Length: Too Short")

if re.search(r"[A-Z]",
password):    print("Uppercase:
```

```

Present") else:

print("Uppercase: Missing")

    if re.search(r"[a-z]",
password):        print("Lowercase:
Present") else:

print("Lowercase: Missing")

    if re.search(r"[0-9]",
password):        print("Number:
Present") else:

print("Number: Missing")

    if re.search(r"^[A-Za-z0-9]",
password):        print("Special Character:
Present") else:    print("Special
Character: Missing")


print("-----
--")    if score <= 2:

        print("Password Strength: WEAK")

elif score <= 4:

        print("Password Strength: MEDIUM")

else:        print("Password Strength:
STRONG")


print("-----
---")

```

11.ADVANTAGES

- Simple and easy to use.
- Beginner-friendly Python implementation.
- Provides immediate feedback.
- Helps users understand password composition.

12.LIMITATIONS

This is an educational checker, not a complete password-auditing system. It does not check password breach databases, predictability, reuse, or advanced attack resistance.

13.CONCLUSION

The Password Strength Checker demonstrates how basic password-security rules can be implemented using Python. The project successfully evaluates common password characteristics and gives a simple strength classification.

14.FUTURE SCOPE

Future versions could include a graphical interface, common-password detection, password entropy estimation, breach-database checking through approved services, and automated unit tests.

END OF TASK 1 REPORT